CS F429 Natural Language Processing — Part 2 Report

Robust Natural Language to SQL Translation Across Single- and Multi-Table Relational Databases

Name: Roshini Kotte | ID Number: 2024A7PS0048H

Abstract

We formulate cross-domain Text-to-SQL and build an execution-based evaluation harness for Spider (multi-table, unseen databases) and WikiSQL (single-table). Two classical baselines are compared: a deterministic template parser and a two-stage pointer-generator BiLSTM sequence-to-sequence model. The headline comparisons use the official Spider evaluator (exact-set match and execution accuracy) with database-clustered 95% confidence intervals, paired McNemar tests and Holm correction. Our own harness supplies the diagnostic breakdowns. Every inference fix was developed on Spider dev and frozen before a single run on the held-out Spider test set (2,147 questions, 40 databases disjoint from train and dev).

On **Spider test**, the template parser reaches **9.5% official execution accuracy** and **6.7% official exact-set match**, against 7.2% and 5.5% for the LSTM. The template's lead on execution accuracy (2.3 points) is significant by McNemar (Holm $p = 0.008$) but borderline under a database-clustered bootstrap ($p = 0.045$ unadjusted, 0.14 adjusted), and the two do not differ significantly on exact-set match. Both baselines almost entirely fail on the stated core objective, multi-table queries (1.83% [0.9%, 3.6%] strict EX for the LSTM on dev). The LSTM's failures are overwhelmingly schema-linking errors: identifier recall is 40.4%, 690 of 746 invalid queries reference a nonexistent column, and its copy gate switches to copying correctly but points at the wrong schema element. Per-epoch dev-loss curves from replication runs of the same recipe show overfitting after about three Spider epochs, although training continued for 25. On WikiSQL we found and fixed a harness defect: gold SQL named columns that the database stores as `col0…colN`, so 71% of gold queries silently returned nothing. With it fixed, the Stage-1 LSTM's near-total failure turns out to be a rendering defect rather than catastrophic forgetting, and fixing the rendering raises its execution accuracy from 5.76% to 17.15%, far above the template's 2.65% (McNemar $p < 10^{-200}$).

a Introduction to the NLP Task

a.1 Motivation

Relational databases hold most structured enterprise and scientific data. Writing SQL against them requires knowing SQL syntax, relational operations (projection, selection, grouping, joins) and the exact table and column names of each schema. Text-to-SQL removes this barrier by translating a natural-language question into an executable query, giving non-technical users conversational access to structured data.

a.2 Core challenges

1. **Schema linking.** Questions rarely use exact identifiers. In *"Which airports are in Los Angeles?"*, the phrase "airports" must be linked to table `airports`, "Los Angeles" to a value of column `City`, and the answer to `AirportName`.
2. **Compositional structure.** Multi-table queries need joins along foreign keys, nested subqueries, `GROUP BY/HAVING`, ordering and set operations.

3. **Cross-domain generalisation.** Test databases are unseen during training, so the model cannot memorise identifiers. It must read the schema supplied at inference time.
4. **Executable validity.** A single unknown column or unquoted literal makes SQLite reject the query, so a nearly-correct token sequence can still score zero.

a.3 Formulation

Given a question $Q = (q_1, \dots, q_m)$ and a schema $\mathcal{S} = (\mathcal{T}, \mathcal{C}, \mathcal{E})$ of tables, columns and foreign-key edges, produce SQL $Y = (y_1, \dots, y_n)$ such that executing Y on the database returns the intended result:

$$\hat{Y} = \arg\max_{Y} \prod_{t=1}^{n} P(y_t \mid y_{<t}, Q, \mathcal{S}).$$

b Literature Survey: Why the Problem Matters

b.1 Rule-based semantic parsing

PRECISE (Popescu et al., 2003) maps words to database elements through bipartite matching and guarantees precision on the questions it accepts, but its recall drops sharply outside its patterns. SCFG (Wong and Mooney, 2007) and CCG (Zettlemoyer and Collins, 2005) parsers induce logical forms but need domain lexicons and do not transfer across schemas.

b.2 Neural sequence-to-sequence parsing

Encoder–decoder LSTMs (Hochreiter and Schmidhuber, 1997; Sutskever et al., 2014) with attention (Bahdanau et al., 2015) treat Text-to-SQL as translation. Two weaknesses matter here. First, identifiers and literals are often out of vocabulary. Second, unconstrained decoders emit ungrammatical or schema-inconsistent SQL.

b.3 Pointer and copy mechanisms

Pointer Networks (Vinyals et al., 2015) and Pointer-Generator Networks (See et al., 2017) mix a vocabulary distribution with a copy distribution over the input:

$$P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i : x_i = w} a_i^t.$$

On WikiSQL, Seq2SQL (Zhong et al., 2017) and SQLNet (Xu et al., 2017) combine pointing with sketch-based slot filling.

b.4 Spider and its baselines

Spider (Yu et al., 2018) has 10,181 questions and 5,693 unique queries over 200 databases in 138 domains, split so that test databases are unseen. **Table 2 of Yu et al. (2018)** reports exact-matching accuracy on the *database split*:

Table 1: Exact-match accuracy of the original Spider baselines (Yu et al., 2018, Table 2, database split).

Model (Yu et al., 2018, Table 2, database split)	Test EM	Dev EM
Seq2Seq	3.7%	1.9%
Seq2Seq + Attention	4.8%	1.8%
Seq2Seq + Copying	5.3%	4.1%
SQLNet	12.4%	10.9%

Model (Yu et al., 2018, Table 2, database split)	Test EM	Dev EM
TypeSQL	8.2%	8.0%

These numbers come from the original 2018 release and evaluator, which ignored literal values. They give an order-of-magnitude reference, not a like-for-like comparison. Later systems close the gap through relation-aware schema encoding (RAT-SQL; Wang et al., 2020), constrained decoding (PICARD; Scholak et al., 2021), and schema ranking with pre-trained seq2seq models (RESDSQL; Li et al., 2023). Our Part 2 model belongs to the Seq2Seq+Copying family.

c Dataset and Preprocessing

c.1 Datasets and splits

Table 2: Datasets used in this work.

	WikiSQL (Zhong et al., 2017)	Spider (Yu et al., 2018)
Domain	Wikipedia tables (open domain)	138 domains, e.g. college, business, entertainment, government
Source	`github.com/salesforce/WikiSQL` (BSD-3)	`yale-lily.github.io/spider` (CC BY-SA 4.0)
Role	Stage-1 warm-up and single-table sanity check	Primary cross-domain benchmark
Size (release files)	80,654 questions; 26,531 table entries in `*.tables.jsonl` (the paper reports 24,241 tables)	7,000 train / 1,034 dev / 2,147 test questions
Databases	One table per question	140 train / 20 dev / 40 test databases, pairwise disjoint (verified)
Operations	`SELECT`, one aggregate, `AND` conditions	Joins, `GROUP BY`, `HAVING`, `ORDER BY`, `LIMIT`, nesting, set operations

Why these datasets. Spider is the standard cross-domain multi-table benchmark: its test databases are unseen, which matches our objective. Every question ships with an executable SQLite database, so execution accuracy can be measured. WikiSQL is large (56k training questions) and simple, which makes it a cheap warm-up and a single-table sanity check for the pipeline.

Per-split retention. All splits are loaded through the typed loaders in `data/spider.py` and `data/wikisql.py`. These record an explicit exclusion event for any example with a missing question, gold SQL, schema or database file.

Table 3: Per-split retention after loading.

Split	Source	Retained	Excluded (reason)
WikiSQL train	56,355	55,968 used in Stage 1	387 unused: 355 beyond `--train-limit 56000` and 32 excluded by the single-table filter

Split	Source	Retained	Excluded (reason)
WikiSQL dev	8,421	8,415	6 (`unsupported_query_structure`: the loader's single-table filter rejected them)
Spider train	7,000	7,000	0
Spider dev	1,034	1,034	0
Spider test	2,147	2,147	0

Spider difficulty distribution (official `eval_hardness`, reimplemented in `data/spider.py` and cross-checked against the official evaluator's counts): train has 1,694 easy, 2,777 medium, 1,461 hard and 1,068 extra-hard examples (3,609 single-table, 3,004 multi-table, 387 nested); dev has 248 easy, 446 medium, 174 hard and 166 extra-hard examples; test has 470 easy, 857 medium, 463 hard and 357 extra-hard (1,100 single-table, 908 multi-table, 139 nested).

Query-structure categories (`_query_structure` in `data/spider.py`) are rule-based on the gold SQL text. A query goes to *multi-table* if it contains `JOIN`, `UNION`, `INTERSECT` or `EXCEPT`. Otherwise it goes to *other (nested)* if it contains more than one `SELECT`, and otherwise to *single-table*. The first matching rule wins, so a query with both a join and a subquery is counted as multi-table, and set operations count as multi-table even over one table. Dev has 544 single-table, 438 multi-table and 52 nested examples.

c.2 Schema serialisation

`data/schema.py` serialises each schema deterministically, with tables and columns sorted:

```
database: concert_singer
table: singer
columns:
  - Age [number]
  - Country [text]
  - Singer_ID [number] [PRIMARY KEY]
foreign_keys:
  - singer_in_concert.Singer_ID -> singer.Singer_ID
```

The encoder input is `<bos> question <schema_sep> schema <eos>`, where both parts are tokenised by the regex lexer in `lstm/preprocessing.py`. Column *types* (`text`, `number`) and markup words (`table`, `columns`) become ordinary copyable source tokens. §d.4 and §d.9 show this was the largest single source of invalid SQL.

c.3 Tokenisation and vocabulary

- **Lexer.** It keeps SQL punctuation and operators, splits `t1.col` into `t1 . col`, upper-cases SQL keywords and lower-cases everything else. A quoted literal becomes one unquoted, lower-cased token (`'United States'` → `united states`).
- **Vocabulary.** Every token in the Stage-1 and Stage-2 *training* questions, schemas and SQL, with no frequency threshold: **102,463 types** (`training_meta.json`). Dev and test are never read when building the vocabulary.
- **Copy targets.** Each example extends the vocabulary with its out-of-vocabulary source tokens. A gold SQL token resolves to the fixed vocabulary first, then to a source copy, else to `<unk>`.

c.4 Worked preprocessing example

This is the output of `scripts/show_preprocessing.py`, which runs the real pipeline on dev example `spider-dev-4`:

```
Question : What is the average, minimum, and maximum age of all singers from France?
Tokens   : what is the average minimum and maximum age of all singers from france   (13)
Schema   : database: concert_singer / table: concert / columns: - Stadium_ID [text] ...
           (34 lines, 4 tables)
Encoder  : <bos> what is ... france <schema_sep> database concert_singer table concert
           columns ... <eos>   (93 tokens)
Gold SQL : SELECT avg(age) , min(age) , max(age) FROM singer WHERE country = 'France'
Target   : SELECT AVG ( age ) , MIN ( age ) , MAX ( age ) FROM singer WHERE country = france
           (every token resolves to the fixed vocabulary; 6 source tokens are copy-only)
```

Three effects are visible. Punctuation is dropped from the question. The literal `'France'` loses its quotes and its case: it becomes the token `france`, so the decoder can only emit `france`. That is why §d.4 restores the surface form from the question at inference. The schema's column *types* (`text`) sit in the encoder input right next to the column names.

c.5 Corpus statistics (as tokenised for the model)

Table 4: Spider corpus statistics as tokenised for the model.

Split	Examples	DBs	Question tokens	SQL tokens	Encoder tokens (max; > 256)	Tables / columns per schema	Target tokens: vocab / copy / unk
Train	7,000	140	12.8	21.7	136 (899; 556)	6.0 / 32.5	100.0% / 0.0% / 0.0%
Dev	1,034	20	12.4	21.5	104 (242; 0)	4.5 / 24.8	90.9% / 8.7% / 0.4%
Test	2,147	40	12.5	22.6	100 (278; 99)	4.9 / 20.9	91.8% / 7.7% / 0.5%

No training target requires copying. The vocabulary contains every training token, so on Spider train every target token can also be *generated* from the fixed vocabulary. On dev and test, 8.7% and 7.7% of target tokens can *only* be produced by copying: unseen table names, column names and values. Is the copy path still learned? The gate still receives gradient, because copying an in-vocabulary source token also produces the correct token. We measured this directly (`scripts/measure_copy_gate.py`, 200 dev questions, teacher-forced). At positions whose gold token is copy-only, the gate chooses to copy ($p_{\text{gen}} < 0.5$) 96.0% of the time (mean p_{gen} 0.04, vs 0.82 for in-vocabulary targets). But it puts only 0.17 probability on the correct source token, vs 0.82 for in-vocabulary targets. The pointer mechanism therefore works as a switch: the model knows *when* to copy, but not *what* to copy. This is the schema-linking failure seen throughout §d.9.

c.6 Dataset limitations

Spider. Most queries appear with two paraphrased questions, which inflates in-domain accuracy. This does not leak across our database-disjoint splits, but it matters for the small live training demo (§d.12). Coverage is English-only and skewed towards college, business and entertainment schemas, and dev has only 20 databases, so per-database variance is high. Small databases allow coincidental execution matches (§d.9 shows one). Gold SQL follows inconsistent conventions: `"blues"` vs `'France'` quoting, stray trailing semicolons and mixed identifier case. These break string match but not execution or exact-set match. One database (`wta_1`) stores non-UTF-8 text. Difficulty is imbalanced: 43% of dev is medium and only 16% is extra-hard.

WikiSQL. Each question targets a single table with at most one aggregate and `AND`-only conditions. Values are stored as text, column names contain spaces and punctuation, and the annotations are known to be noisy. It therefore tests pipeline mechanics rather than compositional parsing.

d Method, Experiments and Results

d.1 Baseline 1: deterministic template parser

`baselines/template.py` matches question tokens to table and column names, detects numeric and quoted literals, comparison phrases, aggregation cues (`how many` → `COUNT`, `average` → `AVG`), `DISTINCT` and ordering cues, and fills a single-table template. It never generates joins. When it finds no confident single-table interpretation it **abstains** with an empty string. In the current code it abstains on **92 of 1,034** dev questions (8.9%) and 208 of 2,147 test questions. Every abstention counts as invalid SQL. Identifiers are double-quoted only when necessary, because the official evaluator's parser reads `"singer"` as a string value. Apart from abstentions the template produced no SQLite errors, so its low invalid rate reflects conservative, single-table output.

d.2 Baseline 2: pointer-generator BiLSTM

`lstm/model.py` (values from `TrainingConfig` and `training_meta.json`):

- **Encoder:** 2-layer bidirectional LSTM. Embedding 256, hidden 256 per direction, so 512-d concatenated states. Packed sequences. Training inputs are truncated at 256 tokens, which cuts 556 of the 7,000 Spider training inputs and hides their alphabetically-last tables. Inference does not truncate: no dev input exceeds 242 tokens, but 99 of 2,147 test inputs do (maximum 278). This train/inference mismatch is a known limitation.
- **Decoder:** 2-layer LSTM with hidden size 512, initialised from projected final encoder states. The input is $[\mathrm{emb}(y_{t-1});\, c_{t-1}]$.
- **Attention:** additive, $e_{t,i} = v^\top \tanh(W_h h_i + W_s s_t)$.
- **Copy gate:** $p_{\mathrm{gen}} = \sigma(W[s_t; c_t; \mathrm{emb}(y_{t-1})] + b)$, mixing P_{vocab} with the attention-weighted copy distribution as in §b.3.
- **Embeddings:** word-level, randomly initialised and trained end to end (no pre-trained embeddings, nothing frozen). The encoder and decoder have separate 256-d embedding tables over the shared 102,463-token vocabulary.
- **Dropout** 0.2. **Decoding:** greedy, at most 128 tokens, with `<pad>`/`<unk>` masked.

Why these two baselines. The template parser is the classical rule-based reference: lexical matching plus fixed SQL templates, in the spirit of PRECISE. The pointer-generator BiLSTM is the canonical pre-transformer neural parser. It is the Seq2Seq+Copying family that Yu et al. (2018) evaluated on Spider, so it shows what learning brings over rules before any pre-training. Both satisfy the course's definition of a classical (pre-transformer) baseline.

Implementation. Python 3.12, PyTorch 2.14 (CUDA 13 on the HPC, CPU locally), SQLite through Python's `sqlite3` (read-only connections), and the official Spider evaluator for scoring. The environment is pinned with `uv` (`uv.lock`), and 87 `pytest` unit tests cover loaders, preprocessing, model, rendering, metrics and statistics.

d.3 Training (HPC Job 361547)

Hardware: one NVIDIA RTX PRO 6000 Blackwell MPS slice (50%), 4 CPUs and 80 GB RAM. The whole job, training plus evaluation, took 50 min of wall time (04:41 to 05:31 IST).

- **One Adam optimiser** with constant learning rate 10^{-3}, batch 64, gradient-norm clipping at 1.0, teacher forcing ratio 1.0, seed 42 and per-epoch shuffling with seed 42 + epoch. There is no learning-rate warm-up or schedule.

- **Stage 1 (WikiSQL):** 55,968 examples, 10 epochs (8,750 steps). Training NLL fell from 1.723 to 0.098. Saved as `wikisql_checkpoint.pt`.
- **Stage 2 (Spider):** 7,000 examples, 25 epochs (2,750 steps), continuing from Stage 1. Training NLL fell from 1.841 to 0.040. Saved as `spider_checkpoint.pt`.

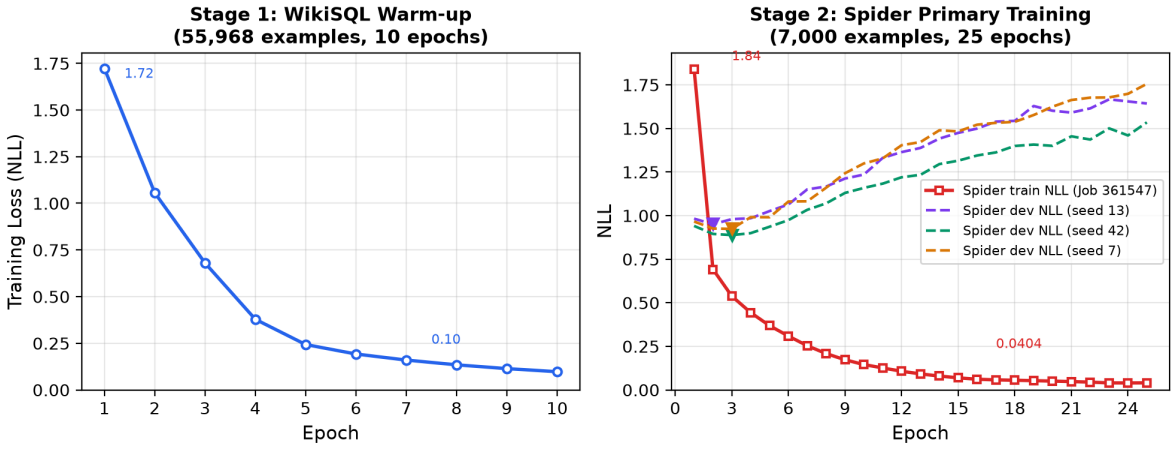


Figure 1: Training NLL (Job 361547) and per-epoch teacher-forced Spider-dev NLL (replication runs, seeds 7, 13, 42; same recipe as Job 361547).

Training dynamics. Training loss alone says nothing about transfer, so the replication runs also log a teacher-forced Spider-dev NLL after every Stage-2 epoch (seed 7: minimum 0.924 at epoch 3, 1.756 at epoch 25; seed 13: minimum 0.950 at epoch 2, 1.643 at epoch 25; seed 42: minimum 0.888 at epoch 3, 1.535 at epoch 25). Dev NLL bottoms out within the first few epochs and then rises steadily while training NLL keeps falling. A training NLL of 0.04 alongside rising dev NLL and over 70% invalid dev SQL indicates **memorisation of the training databases, not successful cross-domain transfer**. Early stopping on a database-disjoint validation split is required for Part 3.

d.4 Inference pipeline

The final pipeline is decode, then render, then repair. All steps operate on the same trained weights.

1. **Schema-markup mask** (`DecodeOptions.mask_schema_markup`), added after inspecting dev errors. Serialisation tokens (`text`, `number`, `time`, `boolean`, `others`, `table`, `columns`, `primary`, `key`, …) are given probability 0 during decoding unless the current schema really has an identifier of that name. With base decoding, the bare token `text` appeared in 278 of 1,034 dev predictions.
2. **Surface-form rendering** (`lstm/rendering.py`), also added after inspecting dev errors. Each decoded token is mapped back to its schema spelling (`School/Club Team` is restored and double-quoted). A value token after a comparison operator becomes a single-quoted literal using the question's original casing (`'France'`, not `'france'`). A multi-word literal stays one literal (`'United States'`, not `'united' states`).
3. **Schema-aware repair** (`lstm/sql_repair.py`, unchanged in substance). It normalises spacing, requotes leftover bare literals, replaces hallucinated `FROM` tables with the schema table sharing the most columns, collapses single-table WikiSQL queries to the sole table, and balances parentheses. It now leaves the inside of quoted literals untouched.

d.5 Evaluation protocol

Official metrics. Spider is scored with the official evaluator (`taoyds/test-suite-sql-eval`, `evaluation.py --etype all`) on Spider's own databases. *Exact-set match* (EM) compares SQL

component by component and ignores literal values. *Execution accuracy* (EX) compares result multisets, and row order matters only when the gold query has `ORDER BY`. Test-suite accuracy (Zhong et al., 2020) needs a separate ~1 GB set of distilled databases, covering Spider dev only. Its official download was over its quota at the time of writing, so it is deferred to Part 3. Execution accuracy can therefore still contain coincidental matches; §d.9 shows one.

Harness metrics. Our own harness (`evaluation/metrics.py`) records four per-example outcomes for every dataset:

- *Strict EX* requires result-column labels (case-insensitive) and row order to agree as well.
- *EX with official semantics* ignores labels and compares rows as multisets.
- *Normalised string match* (SM) is whitespace-collapsed, lower-cased string equality. It is **not** Spider's EM, which explains the template's 0% SM.
- *Invalid SQL* counts any SQLite error or empty prediction, and is decomposed by error class.

Splits and uncertainty. All development decisions were made on Spider dev. **Spider test was run once**, after the configuration was frozen. WikiSQL dev serves as a single-table diagnostic. Questions on the same database are correlated, and Spider pairs most queries with a paraphrase, so confidence intervals come from a bootstrap that resamples whole **databases** (10,000 resamples, seed 42). Paired comparisons report an exact McNemar test (McNemar, 1947) and a paired database-clustered bootstrap (Efron and Tibshirani, 1993). Holm's correction (Holm, 1979) is applied across all comparisons in a table. Per-question official outcomes are produced by `scripts/official_per_example.py`, which mirrors the official scoring loop and reproduces its totals exactly. Training was repeated 3 times (seeds 7, 13, 42) with the identical recipe, as replication runs separate from Job 361547; we report mean ± sample standard deviation across these seeds.

Why also strict EX? Strict EX also requires matching result-column labels, which makes it sensitive to how an expression is spelled. Quoting identifiers alone moved the template's strict dev EX from 5.80% to 7.06%. It is reported only as a secondary diagnostic. All conclusions rest on the official metrics.

Sanity check. Submitting the gold SQL as the prediction gives **100% strict EX** on Spider dev (1,034/1,034) and Spider test (2,147/2,147) and WikiSQL dev (8,415/8,415). This check exposed one harness bug, now fixed: two dev gold queries on `wta_1` crashed on non-UTF-8 text and could never be scored. Gold-equals-gold cannot detect a gold query that runs but means nothing, so we also checked that gold queries return real answers. That exposed a far more serious WikiSQL bug, described in §d.11.

d.6 Main results

Table 5: Main results on Spider test (held out; run once after freezing the configuration). Official metrics with database-clustered 95% CIs. "EM-confirmed EX" counts questions correct under both EX and EM, which excludes coincidental execution matches.

System	Official EX	Official EM	EM-conf. EX	Strict EX	Invalid
LSTM, final decoding	7.2% [5.0%, 10.0%]	5.5% [3.6%, 7.9%]	5.2%	6.52%	72.7%
LSTM, base decoding	6.8% [4.6%, 9.4%]	5.2% [3.4%, 7.6%]	4.8%	6.19%	76.3%
Template parser	9.5% [7.8%, 11.5%]	6.7% [5.1%, 8.5%]	6.6%	7.69%	9.7%

Table 6: Results on Spider dev (development set; all fixes were chosen here).

System	Official EX	Official EM	EM-conf. EX	Strict EX	Invalid
LSTM, final decoding	8.1% [5.0%, 12.1%]	5.0% [3.0%, 7.8%]	5.0%	6.38%	72.1%
LSTM, base decoding	7.4% [4.2%, 11.4%]	4.9% [3.0%, 7.7%]	4.6%	5.90%	77.2%
Template parser	9.9% [6.5%, 13.7%]	5.9% [3.9%, 8.4%]	5.9%	7.06%	8.9%

Table 7: Paired comparisons on the official metrics (difference in points, database-clustered 95% CI; p-values Holm-adjusted within each split).

Split	Comparison	Metric	Difference	McNemar p	Clustered p
test	LSTM final − LSTM base	EX	+0.4 [+0.2, +0.7]	0.047	0.004
test	LSTM final − LSTM base	EM	+0.3 [+0.0, +0.6]	0.089	0.126
test	LSTM final − template	EX	-2.3 [-4.3, -0.1]	0.008	0.136
test	LSTM final − template	EM	-1.2 [-3.2, +1.1]	0.089	0.419
test	LSTM base − template	EX	-2.7 [-4.7, -0.7]	0.001	0.069
test	LSTM base − template	EM	-1.5 [-3.6, +0.8]	0.089	0.419
dev	LSTM final − LSTM base	EX	+0.8 [+0.3, +1.2]	0.047	0.001
dev	LSTM final − LSTM base	EM	+0.1 [+0.0, +0.3]	1.000	0.885
dev	LSTM final − template	EX	-1.7 [-3.8, +0.1]	0.469	0.282
dev	LSTM final − template	EM	-0.9 [-2.7, +0.9]	0.803	0.885
dev	LSTM base − template	EX	-2.5 [-4.5, -0.7]	0.094	0.024
dev	LSTM base − template	EM	-1.0 [-2.8, +0.7]	0.803	0.885

Table 8: Seed variability (replication runs of the same recipe, epoch-25 checkpoint; mean ± sample std over seeds).

Split	Decoding	Official EX	Official EM	Strict EX
Spider test	final	7.6 ± 0.7%	6.0 ± 0.9%	6.7 ± 0.7%
Spider test	base decoding	7.0 ± 0.7%	5.7 ± 0.8%	6.2 ± 0.6%
Spider dev	final	7.5 ± 1.7%	4.9 ± 0.7%	5.9 ± 1.3%
Spider dev	base decoding	6.2 ± 1.9%	4.6 ± 0.7%	5.0 ± 1.5%

n = 3 seeds. The Job 361547 checkpoint (§d.6) is a separate, earlier run of the same recipe.

Across seeds, final-decoding official EX on test is 7.6 ± 0.7% (per seed: 8.0%, 6.8%, 7.9%). Every seed stays below the deterministic template's 9.5%, so the Spider ranking does not depend on the training seed.

Reading the comparison. On Spider the template is the stronger baseline. It leads on official EX on both splits, by 2.3 points on test. This is significant by McNemar after correction ($p = 0.008$), but only borderline once questions are clustered by database ($p = 0.045$ unadjusted, 0.14 adjusted), so it is a probable rather than decisive lead. On official EM the two systems are not significantly different. The decoding fixes give a consistent but small LSTM gain on official EX (+0.4 points on test, clustered $p = 0.004$): statistically real, practically minor. Neither baseline is good at the task. On WikiSQL the ranking reverses (§d.11).

d.7 Inference ablation (same checkpoint, Spider dev)

Table 9: Inference ablation on one checkpoint (Spider dev).

Configuration	Strict EX	Official EX	Official EM	Invalid
Raw greedy tokens, no repair	1.45%	5.4%	4.4%	84.0%
+ repair (**base decoding**)	5.90%	7.4%	4.9%	77.2%
+ surface-form rendering	6.19%	7.6%	4.9%	76.8%
base + schema-markup mask	6.09%	7.7%	5.0%	72.6%
+ mask + rendering (final)	6.38%	8.1%	5.0%	72.1%
final + copy feedback as `<unk>`	6.19%	7.9%	5.0%	72.7%

The CPU re-decode reproduced 1,031 of the 1,034 Job 361547 GPU predictions exactly. The other three are near-tie argmax flips caused by GPU vs CPU floating point. Strict EX is identical (61/1,034), so the *base decoding* row is Job 361547. Repair accounts for most of the gap between *raw* and *base decoding*: strict EX goes from 1.45% to 5.90%. Raw strict EX is especially low because unrepaired spacing changes result-column labels such as `count ( * )`. On official EX, which ignores labels, the effect of repair is much smaller (5.4% → 7.4%). The mask and rendering remove a further 5.0 points of invalid SQL, but most of those queries become *valid but wrong* rather than correct. Example: `WHERE text = 'france'` becomes `WHERE Is_male = 'France'`. Of the 52 queries the mask and rendering made valid, only 5 became correct. Invalid-SQL rate is therefore a misleading optimisation target: the underlying error is schema linking, not syntax. Feeding copied tokens back as `<unk>` did not help and is not used.

Epochs. Our initial 10-epoch run (Job 361487) reached 0.39% SM, and Job 361547, with 25 epochs plus repair, reached 3.29%. That comparison changes several things at once. The table above isolates repair on a single checkpoint. To isolate the epoch count, the replication runs also saved a checkpoint after Spider epoch 10. With identical (final) decoding, epoch 10 vs epoch 25 gives official EX 7.6 ± 1.1% vs 7.5 ± 1.7% on dev and 6.9 ± 0.8% vs 7.6 ± 0.7% on test; official EM 4.8 ± 0.5% vs 4.9 ± 0.7% (dev) and 5.3 ± 0.7% vs 6.0 ± 0.9% (test).

d.8 Breakdown by difficulty and structure (Spider dev, final LSTM)

Table 10: Strict EX by difficulty and query structure (Spider dev, Wilson 95% CI).

Slice	n	LSTM strict EX (95% CI)	Template strict EX (95% CI)
Easy	248	14.92% [11.0%, 19.9%]	21.37% [16.7%, 26.9%]
Medium	446	4.26% [2.7%, 6.6%]	3.81% [2.4%, 6.0%]
Hard	174	5.75% [3.2%, 10.3%]	0.00% [0.0%, 2.2%]
Extra-hard	166	0.00% [0.0%, 2.3%]	1.81% [0.6%, 5.2%]
Single-table	544	10.29% [8.0%, 13.1%]	12.68% [10.1%, 15.7%]
Multi-table (join / set op)	438	1.83% [0.9%, 3.6%]	0.91% [0.4%, 2.3%]
Nested (other)	52	3.85% [1.1%, 13.0%]	0.00% [0.0%, 6.9%]

Multi-table queries, the core objective, are essentially unsolved: 1.83% [0.9%, 3.6%] strict EX for the LSTM, and SM is 0/438 in both the base-decoding and final runs. Single-table queries account for 56/66 (84.8%) of all LSTM EX hits, and extra-hard is 0/166, a floor effect. Hard EX is slightly above medium EX, but this comes from small counts: the 95% intervals, 4.26% [2.7%, 6.6%] for medium and 5.75% [3.2%, 10.3%] for hard, overlap almost entirely, so the ordering should not be interpreted.

d.9 Error analysis (all 1,034 dev questions, final LSTM)

Table 11: Invalid SQL on Spider dev by SQLite error class.

Error class	Base decoding	Final
`no_such_column`	708	690
`syntax_error`	55	29
`ambiguous_column`	6	9
`incomplete_input`	19	8
`no_such_table`	6	5
`aggregate_misuse`	3	4
`other`	1	1
Total invalid	**798**	**746**

Root-cause taxonomy. Each failed prediction is assigned exactly one label: the *first* failing component, in the order invalid SQL → wrong table set → wrong columns → wrong clause structure → wrong aggregation → wrong projection count → wrong values. Labels are assigned automatically by `scripts/analyze_part2.py`, which compares literal-stripped signatures of the predicted and gold SQL. They were not manually validated.

Table 12: Root cause of each failed prediction (Spider dev).

Root cause	Final LSTM	Template
Invalid SQL (or abstention)	746	92
Wrong table set	167	592
Wrong columns	51	242
Wrong clause structure	3	29
Wrong aggregation	0	3
Wrong projection count	1	1
Wrong literal values	0	1
Other (same signature)	0	1
Total failures	**968**	**961**

Schema linking. Identifiers are extracted from each SQL string after removing literals, as the words that match a table or column name of that database (case-insensitive). Tables are the names after `FROM`/`JOIN`. Precision and recall are micro-averaged over all dev questions. For the LSTM, identifier precision/recall is 39.9% / 40.4% and table precision/recall is 46.8% / 46.5%. The template is a purely lexical linker (question words matched to schema names), so it serves as a lexical baseline: 68.5% / 29.3% for identifiers and 69.4% / 41.8% for tables. The LSTM recalls more schema elements than lexical matching but with far lower precision. Choosing the wrong column or table is the dominant failure, ahead of syntax.

Execution-match audit. Execution hits that do not string-match the gold SQL are classified automatically. A hit is *equivalent* when its literal-stripped signature equals the gold one; otherwise it is *coincidental*, split by whether the gold result is empty or zero. Template: 14 structurally equivalent to gold; 13 coincidental (a gold clause is missing or different); 6 coincidental (gold result is empty or zero). LSTM: 15 structurally equivalent to gold; 10 coincidental (gold result is empty or zero); 7 coincidental (a gold clause is missing or different). For example, `SELECT DISTINCT Country FROM singer` matches gold … `WHERE age > 20` only because every singer is over 20. The EM-confirmed EX column of the main results tables is the corresponding conservative accuracy.

How these errors can be mitigated. The recurring pattern is schema grounding, not syntax. The model knows when to copy but picks the wrong table or column, and joins ignore the foreign-key graph. Our analysis

points to five concrete remedies. (i) A pre-trained encoder with explicit question–schema linking (cross-attention, or value matching against database contents), aimed at the 40% identifier recall. (ii) Foreign-key-aware or relation-aware decoding (RAT-SQL style) for join paths. (iii) Grammar- and schema-constrained decoding (PICARD style), so that nonexistent columns and malformed clauses cannot be generated. (iv) A vocabulary that leaves schema identifiers to the copy path, so that copying *what* is learned and not only *when*. (v) Early stopping at the dev-loss minimum, and no input truncation during training. Items (i)–(iii) define Part 3 (§d.13).

d.10 Qualitative cases (Spider dev; base → final decoding)

1. **Correct** (`spider-dev-20`, medium): *"How many concerts are there in year 2014 or 2015?"* → `SELECT COUNT(*) FROM concert WHERE Year = 2014 OR Year = 2015`. Both EM and EX are correct. The numeric literals are in-vocabulary tokens, so this case shows correct generation; the copy gate is analysed separately in §c.5.
2. **Execution-equivalent** (`spider-dev-87`, easy): gold `SELECT count(*) FROM CONTINENTS;` vs `SELECT COUNT(*) FROM continents`. EX is correct, and the official exact-set match also counts it correct. Our string match does not, because of the trailing semicolon.
3. **Projection omission** (`spider-dev-2`, medium): *"Show name, country, age …"* → `SELECT Name, Country FROM singer ORDER BY Age DESC`. Valid SQL; the third projected column is dropped.
4. **Schema-type token leak, then wrong column** (`spider-dev-4`, medium). Base decoding produced `… WHERE text = 'france'`. That query is invalid: `text` is the serialised *type* of `Country`, and the literal case is wrong. The final configuration produces `… WHERE Is_male = 'France'`. Casing and validity are fixed, but the column is still wrong. Rendering cannot fix schema linking.
5. **Aggregation error** (`spider-dev-494`, medium): *"What is maximum and minimum death toll …"* → `SELECT MAX(caused_by_ship_id), MIN(caused_by_ship_id), MIN(killed) FROM death`. The aggregate functions are plausible but attached to the wrong columns, plus an extra projection. Aggregate functions differ from gold in 167 failed predictions; in the root-cause view these failures are attributed to the earlier error, usually a wrong table or column.
6. **Join hallucination** (`spider-dev-24`, extra-hard). Gold joins `concert` and `stadium`. The prediction joins `singer_in_concert` and `singer` on `Singer_ID = concert_ID` and selects the nonexistent `t2.stadium`. It shows correct clause skeleton generation with no grounding in the foreign-key graph.

d.11 WikiSQL: a harness defect and a rendering defect

A natural explanation for the LSTM's 0.04% WikiSQL EX would be catastrophic forgetting during Spider fine-tuning. **The run log rules this out.** Job 361547 scored WikiSQL with `wikisql_checkpoint.pt`, saved at the end of Stage 1, *before any Spider training* (log line: `Loading dataset-specific checkpoint: wikisql_checkpoint.pt`). The predictions are mostly sensible SQL in an unexecutable surface form. For example, `SELECT player FROM "table_1_10015132_11" WHERE no.= 42` should be `… WHERE "No." = 42`. Multi-word column names were emitted unquoted and lower-cased (`school/club team`), and the repair step quoted only the first word of multi-word literals (`= 'concord' hs`). The fault lay in the rendering, not the model.

Harness defect. The official WikiSQL database stores each table's columns as `col0 … colN`, with lower-cased text values. Our loader, however, wrote gold SQL with the original header names, such as `SELECT "Position" … WHERE "School/Club Team" = 'Butler CC (KS)'`. SQLite treats a double-quoted name that matches no column as a *string literal*. Every gold query therefore executed without error but compared constants: 6,007 of 8,415 dev gold queries returned an empty result, and any prediction that also returned nothing counted as correct. All earlier WikiSQL execution numbers, the template's 7.23% included, were therefore meaningless. The harness now creates, per connection, a TEMP VIEW with the table's own name that exposes `colN` under its header. Text columns compare case-insensitively, mirroring the official executor, which lower-cases condition values. With the view, 7,767 gold queries return real answers. The table below re-scores the same predictions with the corrected harness.

Table 13: WikiSQL dev (all 8,415 examples, corrected harness), Stage-1 checkpoint.

System	Strict EX (95% CI)	SM	Invalid
LSTM Stage-1, raw greedy	3.11% [2.8%, 3.5%]	0.00%	89.5%
LSTM Stage-1, base decoding	5.76% [5.3%, 6.3%]	0.04%	81.1%
LSTM Stage-1, final decoding	17.15% [16.4%, 18.0%]	1.84%	32.5%
Template parser	2.65% [2.3%, 3.0%]	1.26%	0.0%

With the same Stage-1 weights, rendering takes strict EX from 5.76% to 17.15% and invalid SQL from 81.1% to 32.5% (958 vs 0 questions gained vs lost). The remaining invalid queries are 1,890 syntax errors and 822 unknown columns. The syntax errors typically come from a hallucinated multi-word column name: one that exists in some training table but not in the current table (e.g. `points for`, `year born`), so it is never quoted. Both are schema-linking failures, as on Spider. Unlike on Spider, the LSTM is clearly the stronger system here: the template's single-table heuristics reach only 2.65%. Its previously measured 7.23% came almost entirely from coincidental empty-result matches under the broken harness. Both systems remain far below published WikiSQL models: Seq2SQL reports 59.4% and SQLNet 68.0% test execution accuracy (Xu et al., 2017).

d.12 Demonstration (CLI)

Live training. `uv run text-to-sql-demo --train-demo` trains a small pointer-generator (embedding 128, hidden 256) from scratch on the 95 Spider *train* questions of `soccer_2`, for 60 epochs on a laptop CPU, which takes about 2–3 minutes. It prints the per-epoch training and held-out NLL. It decodes a probe question with random weights before training and again after. It then reports execution accuracy on 20 questions the model trained on and on 11 held-out questions from the same database. In our rehearsal the loss fell from 4.88 to 0.09, and the model scored 15/20 on seen questions but 0/11 on held-out ones. That is the memorisation gap of §d.3 at small scale. The saved checkpoint can be queried at once with `--model lstm --checkpoint artifacts/train-demo/spider_checkpoint.pt`.

Inference. `--demo --model template` runs a scripted three-case showcase on `concert_singer`: a count, a filter, and a graceful failure on an unsupported join. `--model lstm` loads the evaluated Job 361547 checkpoint with the final decoding configuration. `--question "…" --database <path>` answers any question and prints the SQL and the result table. For Spider databases, the CLI builds the schema from `tables.json`, so the serialisation matches training exactly. `scripts/show_preprocessing.py` prints the pipeline stages of §c.4 for any example. The proposal planned a GUI; the Part 2 interface is this CLI, which the course rubric explicitly allows ("results should be shown through a GUI or CLI").

d.13 Targets for Part 3

The proposal fixes the Part 3 plan: fine-tune **T5** as the modern model, and add a novelty, either *constrained decoding* or a *self-correction loop*, that must improve execution accuracy over that T5 baseline. The targets below are anchored to our measured Part 2 baselines and to published results. All are scored with the official evaluator on Spider test, run once, as mean ± std over at least 3 seeds. Early stopping uses a database-disjoint validation split carved from Spider *train*.

Table 14: Numeric targets for Part 3 (Spider test unless stated).

Goal	Metric	Best Part 2 result	Part 3 target	Reference point
Modern baseline works	Official EX (dev)	9.9% (template)	$\geq 45\%$	T5-Base, schema only: 57.9% dev EX (Scholak et al., 2021, Table 1), trained far longer (up to 3,072 epochs, batch 2,048)
Classical → modern gain	Official EX	9.5% (template); 7.2% (LSTM)	$\geq 35\%$, McNemar $p < 0.05$ vs template	—
Multi-table queries	Strict EX, 908 multi-table questions	0.7% (LSTM); 0.3% (template)	$\geq 25\%$	—
Novelty beats T5 baseline	Official EX	—	Gain with McNemar $p < 0.05$ in every seed	PICARD: T5-Base 57.9% → 68.4% dev EX (Scholak et al., 2021)
Valid SQL (diagnostic only)	Invalid-SQL rate	72.7% (LSTM)	Report, not optimise	PICARD: T5-3B invalid predictions 12% → 2% on dev; §d.7 shows lower invalid rates need not mean higher accuracy
Schema linking	Identifier recall	45.1% (LSTM)	$\geq 75\%$	—

We will also report test-suite accuracy (Zhong et al., 2020) on Spider dev once the distilled test-suite databases can be obtained; the official download was over its quota at the time of writing.

Reproducibility

Artefact	Command
Training (3 seeds)	`sbatch --export=ALL,SEED=<s> scripts/hpc_run_lstm_seed.sh`
Ablation / test decode	`uv run python -m scripts.run_ablation --checkpoint <ckpt> --dataset spider --split {dev,test} --with-template --output-dir <dir>`
Official scoring	`uv run python scripts/run_official_spider_eval.py --predictions <jsonl> --split {dev,test} --output-dir <dir>`
Taxonomy, CIs, McNemar, gold sanity check	`uv run python -m scripts.analyze_part2 --dataset spider --split dev --model name=<jsonl> … --output-dir <dir>`

References

1. Bahdanau, D., Cho, K., and Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR*.
2. Efron, B., and Tibshirani, R. J. (1993). *An Introduction to the Bootstrap*. Chapman & Hall.
3. Hochreiter, S., and Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation* 9(8).
4. Holm, S. (1979). A Simple Sequentially Rejective Multiple Test Procedure. *Scandinavian Journal of Statistics* 6(2).
5. Li, H., Zhang, J., Li, C., and Chen, H. (2023). RESDSQL: Decoupling Schema Linking and Skeleton Parsing for Text-to-SQL. *AAAI*.
6. McNemar, Q. (1947). Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika* 12(2).
7. Popescu, A.-M., Etzioni, O., and Kautz, H. (2003). Towards a Theory of Natural Language Interfaces to Databases. *IUI*.
8. Scholak, T., Schucher, N., and Bahdanau, D. (2021). PICARD: Parsing Incrementally for Constrained Auto-Regressive Decoding from Language Models. *EMNLP*.
9. See, A., Liu, P. J., and Manning, C. D. (2017). Get To The Point: Summarization with Pointer-Generator Networks. *ACL*.
10. Sutskever, I., Vinyals, O., and Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS*.
11. Vinyals, O., Fortunato, M., and Jaitly, N. (2015). Pointer Networks. *NeurIPS*.
12. Wang, B., Shin, R., Liu, X., Polozov, O., and Richardson, M. (2020). RAT-SQL: Relation-Aware Schema Encoding and Linking for Text-to-SQL Parsers. *ACL*.
13. Wong, Y. W., and Mooney, R. J. (2007). Learning Synchronous Grammars for Semantic Parsing with Lambda Calculus. *ACL*.
14. Xu, X., Liu, C., and Song, D. (2017). SQLNet: Generating Structured Queries from Natural Language Without Reinforcement Learning. *arXiv:1711.04436*.
15. Yu, T., et al. (2018). Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task. *EMNLP*.
16. Zettlemoyer, L. S., and Collins, M. (2005). Learning to Map Sentences to Logical Form. *UAI*.
17. Zhong, V., Xiong, C., and Socher, R. (2017). Seq2SQL: Generating Structured Queries from Natural Language using Reinforcement Learning. *arXiv:1709.00103*.
18. Zhong, R., Yu, T., and Klein, D. (2020). Semantic Evaluation for Text-to-SQL with Distilled Test Suites. *EMNLP*.